Waseet — Frontend Implementation Prompt

Oct 20, 2026 · @Osama Riyad

A paste-ready brief for a coding agent: hand it this whole document to build the Waseet frontend. It embeds the UI-relevant SRS, the design direction, the backend API contract, and the tech stack, plus the frontend requirements and a per-screen checklist. The frontend consumes the separate Waseet backend API.

Role & project

Act as a Senior Frontend Engineer and UI Architecture Expert. Build the frontend for the following software, using this document as the complete and primary brief.

Project — Waseet: an AI-mediated escrow service on PayPal. A buyer funds a deal through PayPal; the platform holds the funds in escrow; an AI verifies the seller's deliverable against agreed acceptance criteria and auto-releases payment on a confident pass, or routes the deal to a human-reviewed dispute. The frontend is a two-sided web app (buyer and seller see the same deal, different actions) plus a reviewer view for disputes. It consumes the separate Waseet backend API; all payments run against the PayPal **sandbox**.

SRS (embedded)

The UI-relevant requirements. Treat them as the source of truth; every workflow below must have a frontend flow.

ID	Requirement (UI-facing)
FR-1	Create a deal from a plain-language description (buyer, seller, amount, currency, description).
FR-2	Show the AI-drafted acceptance checklist; both parties confirm before funding.
FR-3	Buyer funds the deal via PayPal checkout; show the deal move to FUNDED.
FR-4	Seller submits a deliverable (URL, file, image, repo link, or text).
FR-5	Show the AI verification report: per-criterion pass/fail, verdict, confidence, reasoning.

ID	Requirement (UI-facing)
FR-6 / FR-7	Reflect auto-release on a confident pass, or a move to DISPUTED otherwise.
FR-8	Reviewer sees the AI recommendation and chooses release / refund / split.
FR-11	Both parties read a deal timeline: current state, verification report, dispute outcome.
FR-12	Reflect key events (funded, delivered, verified, released, refunded, disputed) in near-real-time.

Lifecycle states to display: DRAFT, FUNDED, DELIVERED, VERIFYING, RELEASED, REFUNDED, DISPUTED, SPLIT. The current state drives the status badge and the available actions.

User workflows the frontend must cover:

1. Buyer and seller agree a deal → review and confirm AI criteria → buyer funds.
2. Seller submits a deliverable → app shows VERIFYING → shows the verification report.
3. Confident pass → auto-release shown; seller sees funds released.
4. Fail / low confidence / buyer contest → dispute opened → reviewer resolves → outcome shown.
5. Either party opens the deal timeline and reads the full history and audit trail.

UI / design references

Direction: trust-forward, status-first. Escrow is an anxiety product — both sides are asking "where's my money, and is it safe?" The hero is the deal timeline with a large, color-coded status, not a dense dashboard. Calm, high-contrast, generous whitespace, one confident accent, large legible money numbers. Ship RTL with an Arabic/English toggle from day one.

Design tokens — status color system (used everywhere status appears):

State	Meaning	Color role
FUNDED / DRAFT	Held / pending	Neutral blue
VERIFYING / DELIVERED	In progress	Muted accent
RELEASED / SPLIT	Paid out	Green

State	Meaning	Color role
REFUNDED	Returned	Amber
DISPUTED	Needs attention	Red

Core reusable components: `StatusBadge`, `MoneyAmount` (currency-aware), `DealTimeline` (the hero), `CriteriaChecklist` (confirmable), `DeliverableUploader` (file/URL/text), `VerificationReport` (per-criterion pass/fail + confidence + reasoning), `DecisionPanel` (reviewer: release/refund/split), `PayPalCheckoutButton`.

Screens: (1) Login, (2) Deals list, (3) Create deal + criteria review, (4) Deal timeline (hero), (5) Submit deliverable (seller), (6) Verification report, (7) Dispute / reviewer decision.

Reference products to mirror: Escrow.com (flow), Upwork/Fiverr milestone-release (two-sided pattern), Stripe Dashboard (clean money + timeline), Wise (money-movement clarity), Linear (state/timeline legibility). Browse Mobbin, Dribbble and Figma Community for "escrow", "milestone payment", "transaction timeline".

Replace this section with a real Figma link or screenshots if available; otherwise build to this direction.

Backend / API contract

REST over HTTPS; JSON; JWT bearer auth on all endpoints except login. Replace with the backend's real OpenAPI spec once generated; this is the expected shape.

Method	Path	Purpose	Returns
POST	/auth/login	Authenticate; issue a bearer token	token, user (id, role)
GET	/deals	List the current user's deals	array of deal summaries
POST	/deals	Create a deal from a description	deal + AI-drafted criteria
GET	/deals/{id}	Full deal: state, criteria, deliverable, verification, timeline	deal detail
POST	/deals/{id}/criteria/confirm	Confirm acceptance criteria (per party)	updated criteria

Method	Path	Purpose	Returns
POST	/deals/{id}/order	Create a PayPal order for funding	paypal_order_id
POST	/deals/{id}/capture	Capture the order into escrow	deal (state = FUNDED)
POST	/deals/{id}/deliverable	Submit a deliverable	deliverable
POST	/deals/{id}/verify	Trigger AI verification	verification report
GET	/deals/{id}/verification	Fetch the latest verification report	report
POST	/deals/{id}/dispute/resolve	Reviewer decision (release/refund/split)	deal (terminal state)
GET	/deals/{id}/audit	Append-only audit trail for the timeline	array of audit entries

Funding uses the PayPal JS SDK on the client to render checkout; the resulting order id is sent to `/deals/{id}/capture`, which captures server-side. Treat state as owned by the backend — after any action, refetch the deal rather than guessing the next state locally.

Tech stack

Concern	Choice
Framework	Next.js (App Router) + TypeScript
Styling	Tailwind CSS + shadcn/ui
Server/API state	TanStack Query (React Query)
Forms + validation	react-hook-form + zod
Animation	Framer Motion (status transitions, the release moment)
Payments	PayPal JavaScript SDK v6
i18n / RTL	next-intl (or equivalent), Arabic + English, RTL-aware
Tests	Vitest + React Testing Library; Playwright for the key flows

Concern	Choice
Deploy	Render (or Vercel)

Rationale: TanStack Query keeps server state (deal, verification, audit) separate from view components and gives loading/error/refetch for free; react-hook-form + zod cover form validation consistently; shadcn/ui provides accessible, themeable primitives so design tokens stay consistent. Use the PayPal **JS SDK v6**, not v5.

Requirements

1. Build the UI according to the product requirements and the design direction above.
2. Create a clear component architecture (reusable primitives → feature components → screens).
3. Separate presentation from business/state logic — data fetching in hooks, visuals in components.
4. Implement loading, empty, error, and success states for every data-driven screen.
5. Implement form validation (react-hook-form + zod) with inline, accessible error messages.
6. Handle API failures gracefully — retry where safe, clear error UI, never a blank screen.
7. Implement authentication and protected routes — redirect unauthenticated users; gate actions by role.
8. Maintain consistent design tokens and reusable components — one status color system, one MoneyAmount.
9. Ensure responsive behavior — the deal timeline works at phone width with no horizontal scroll.
10. Consider accessibility — semantic markup, focus states, labels, color is never the only status signal.
11. Avoid duplicated UI logic — shared hooks (useDeal, useVerification) and shared components.
12. Keep API integration separate from visual components — a typed API client + query hooks, not fetch calls inside JSX.

Screens & features

For each screen: identify its responsibility, required components, state, API calls, user interactions, and edge states (loading / empty / error / success). Build to this breakdown.

Screen	Responsibility	Key components	State	API calls	Primary interactions
Login	Authenticate, store token, route by role	AuthForm	form, auth token	POST /auth/login	Submit credentials
Deals list	Show the user's deals by status	DealCard, StatusBadge, Filters	deals query, filter	GET /deals	Open a deal, filter by state
Create deal + criteria	Capture the deal, review AI criteria, confirm	DealForm, CriteriaChecklist	form, criteria, confirm flags	POST /deals, POST /deals/{id}/criteria/confirm	Enter terms, confirm criteria
Deal timeline (hero)	Show state, amount held, history; surface the next action	DealTimeline, StatusBadge, MoneyAmount, action buttons	deal query, audit query	GET /deals/{id}, GET /deals/{id}/audit	Fund, submit, open dispute, read history
Submit deliverable	Let the seller submit work	DeliverableUploader	upload state, form	POST /deals/{id}/deliverable, POST /deals/{id}/verify	Upload file/URL/text, submit
Verification report	Show per-criterion verdict + reasoning + decision	VerificationReport, ConfidenceMeter	verification query	GET /deals/{id}/verification	Read report, contest (buyer)
Dispute / reviewer	Show AI recommendation, let reviewer decide	DecisionPanel, VerificationReport	deal query, decision form	POST /deals/{id}/dispute/resolve	Choose release / refund / split
Funding (modal/step)	Run PayPal checkout, capture into escrow	PayPalCheckoutButton	order id, capture status	POST /deals/{id}/order, POST /deals/{id}/capture	Complete PayPal checkout

Edge states to implement everywhere: a VERIFYING screen (not a frozen UI) while the model runs; an empty state for a buyer or seller with no deals; a clear error + retry on any failed fetch or payment; an optimistic-but-verified success (refetch the deal to confirm the new state rather than assuming it).

Final verification

Before finalizing, verify that every user workflow in the SRS has a corresponding frontend flow:

SRS workflow	Frontend flow
Agree deal → confirm criteria → fund	Create deal + criteria review → funding (PayPal) → timeline shows FUNDED
Submit deliverable → verify	Submit deliverable → VERIFYING state → verification report

SRS workflow	Frontend flow
Confident pass → release	Timeline shows RELEASED; seller sees payout confirmation
Fail / low confidence / contest → dispute → resolve	DISPUTED state → reviewer DecisionPanel → outcome on the timeline
Read full history	Deal timeline + audit trail

Report any workflow without a flow, any screen missing a loading/empty/error/success state, and any assumption made where the design or API was unspecified (for example: token storage, how contest is triggered, RTL fallbacks). Do not add screens beyond the SRS unless marked optional.